

Data Structures

Unit- 4

Tree

Linear vs Non-linear Data structure

- A linear data structure is a structure in which the elements are stored sequentially, and the elements are connected to the previous and the next element.
- traversed or accessed in a single run.
- The implementation is easier

Linear vs Non-linear Data structure

- A non-linear data structure is also another type of data structure in which the data elements are not arranged in a contiguous manner.
- traversed or accessed not in a single run.
- In the case of linear data structure, element is connected to two elements (previous and the next element), whereas, in the non-linear data structure, an element can be connected to more than two elements.

Basic Tree Concepts

- A tree consists of a finite set of elements, called **nodes** and a finite set of directed lines, called **branches**, that connect the nodes.
- The number of branches associated with a node is the **degree** of the node.
- When the branch is directed towards the node, it is an indegree branch.
- When the branch is directed away from the node, it is an outdegree branch.

- **Indegree of a node-**

Number of branches directed towards that node is called indegree of that node.

- **Outdegree of a node-**

Number of branches directed away from that node is called outdegree of that node.

- The sum of the indegree and outdegree branches is degree of the node.

Tree Terminology

- Root

If the tree is not empty, the first node is called the root of tree.

- Indegree of the root is, by definition zero
- Outdegree of the root may vary depending on type of tree.
- Every node in a tree can have zero, one or more successor.
- Every node in a tree can have zero, one or more outdegree.

- All the nodes in a tree, except root must have an indegree of exactly one.
- i.e. every node except root in a tree have only one predecessor.

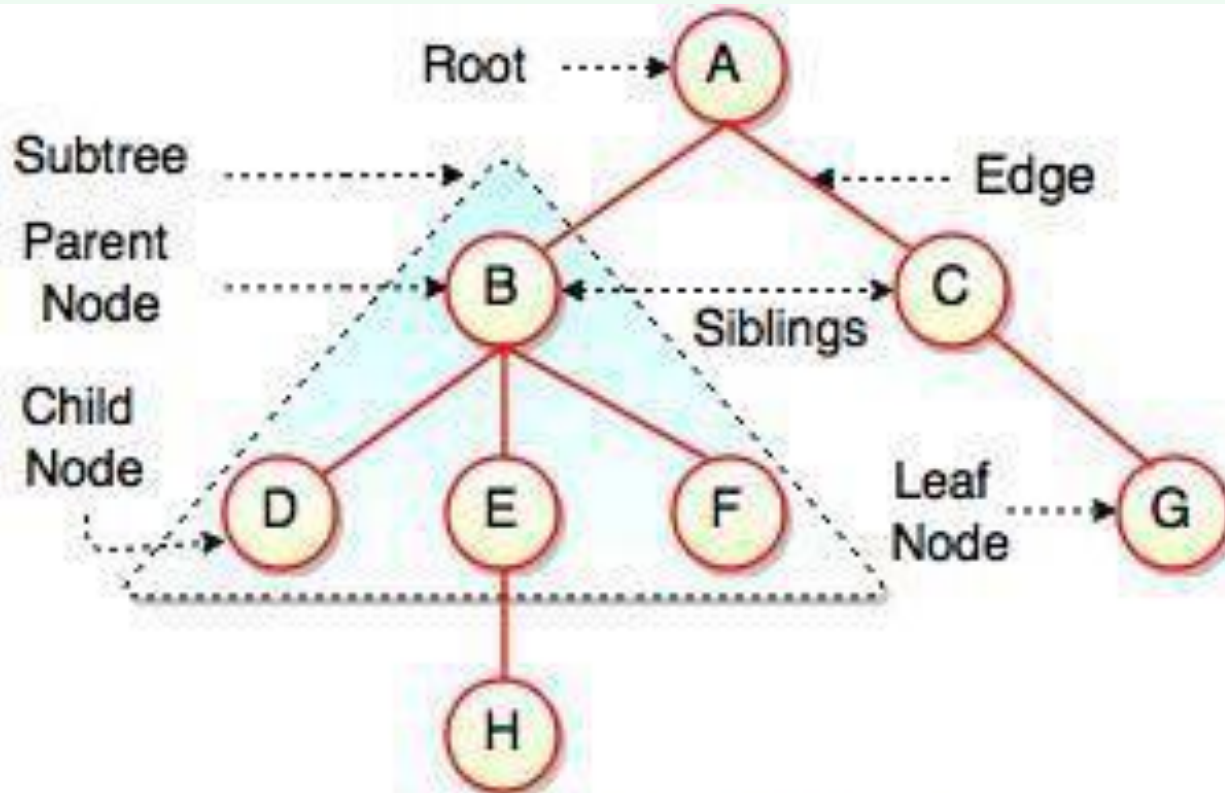


Fig. Structure of Tree

- Leaf

Any node in a tree, with zero outdegree. A node in tree with no successor.

- Internal Node

A node that is not a root or a leaf is known as internal node.

- **Parent Node**

A node is a parent if it has successor nodes.

A node having outdegree greater than zero is a parent node.

- **Child Node**

A node with predecessor is called a child.

A child node has indegree of one.

- **Siblings**

Two or more nodes with the same parents are called siblings.

- Ancestor

An ancestor is any node in the path from the root to the node.

- Descendent

A descendent is any node in the path below the parent node.

All nodes in the paths from a given node to a leaf are descendants of that node.

- Path

A path is a sequence of nodes in which, each node is adjacent to the next one.

- Every node in the tree can be reached by following a unique path starting from the root.

- Level of a node

Level of node is its distance from the root.

- Root is at level 0.
- Children of a root are at level 1.
- Siblings are always at same level.
- But not all nodes at same level are siblings.

- Height of the tree / Depth of the tree

The height of the tree is the level of the leaf in the longest path from root plus 1.

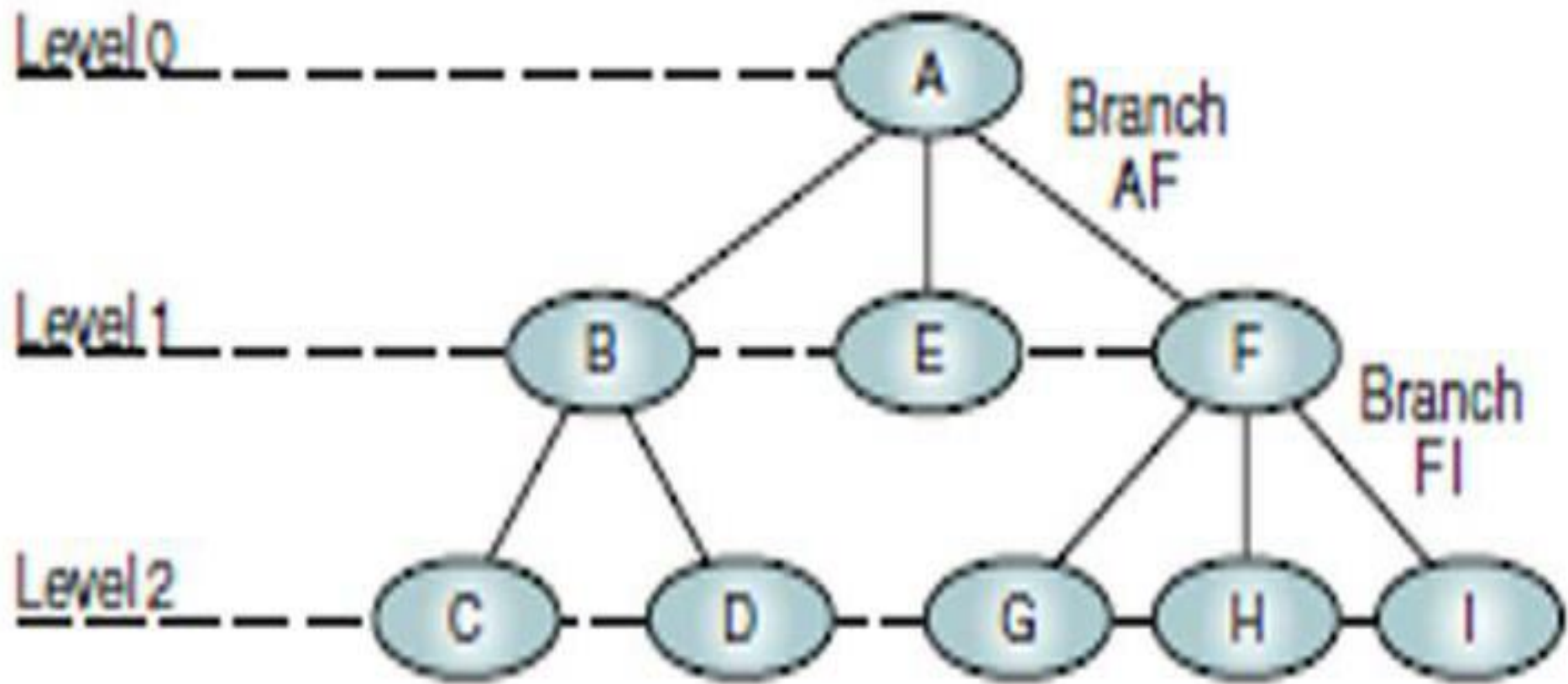
- Height of an empty tree is -1.

- Subtree

A tree may be divided into subtrees.

A subtree is any connected structure below the root.

- The first node in a subtree is known as the root of the subtree and it is used to name the subtree.
- A subtree can further sub divided into subtrees.



Root: A
Parents: A, B, F
Children: B, E, F, C, D, G, H, I

Siblings: {B, E, F}, {C, D}, {G, H, I}
Leaves: C, D, E, G, H, I
Internal nodes: B, F

- Recursive Definition of Tree

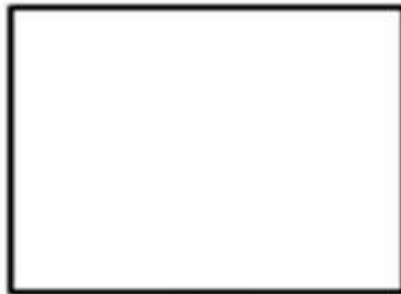
A tree is a set of nodes that either

1] is empty or

2] has a designated node, called the root, from which hierarchically descend zero or more subtrees, which are also trees.

Binary Trees

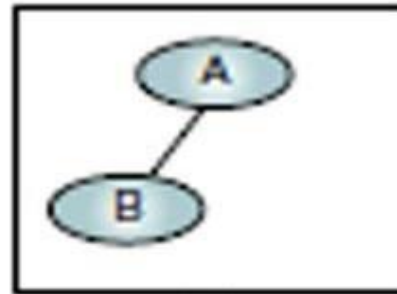
- A binary tree is a tree in which no node can have more than two sub-trees.
- Maximum out-degree for a node is two.
- A node can have zero, one or two sub-trees.
- These sub-trees are designated as left sub-tree and right sub-tree.
- A null tree is a tree with no nodes.
- Symmetry is not a requirement of tree.



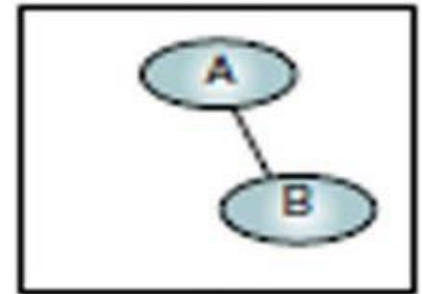
(a)



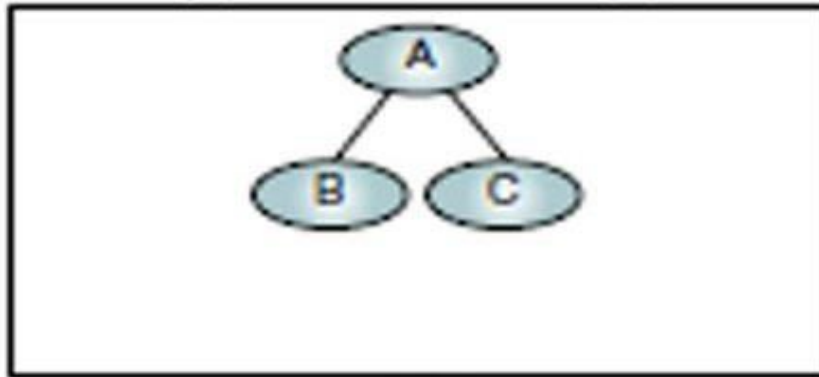
(b)



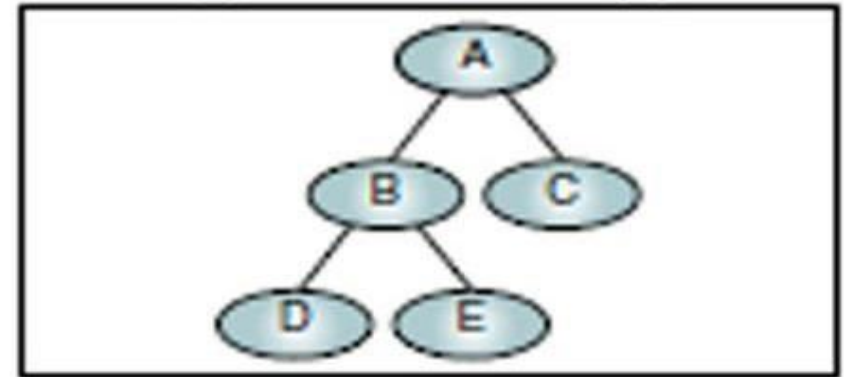
(c)



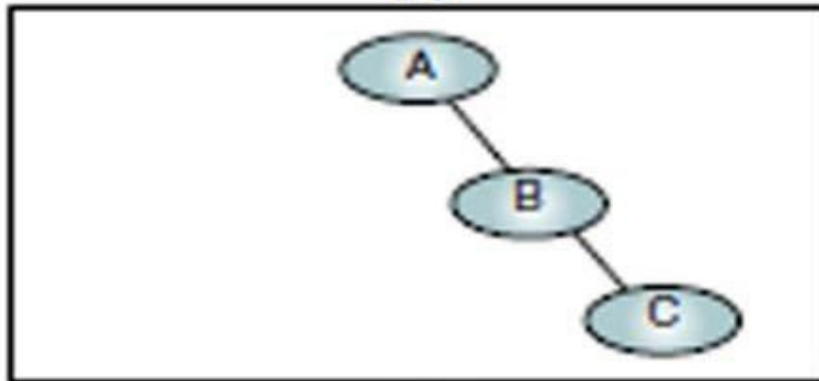
(d)



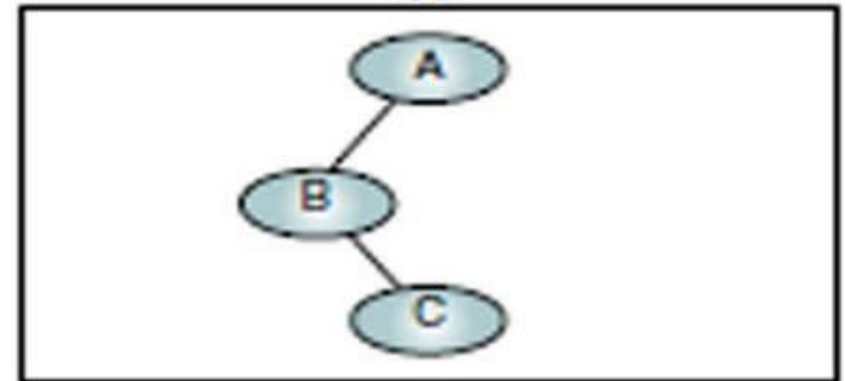
(e)



(f)



(g)



(h)

Properties of Binary Tree

- The properties distinguish binary trees from general trees.
- Height of Binary Trees
- Balance

- Maximum height of Binary Tree with N nodes
- If N nodes in binary tree, then
- Maximum height = $H_{\max} = N$
- All the nodes in a tree have only one successor.
- Minimum height of Binary Tree with n nodes
- If N nodes in binary tree, then
- Minimum height = $H_{\min} = \log_2 N + 1$

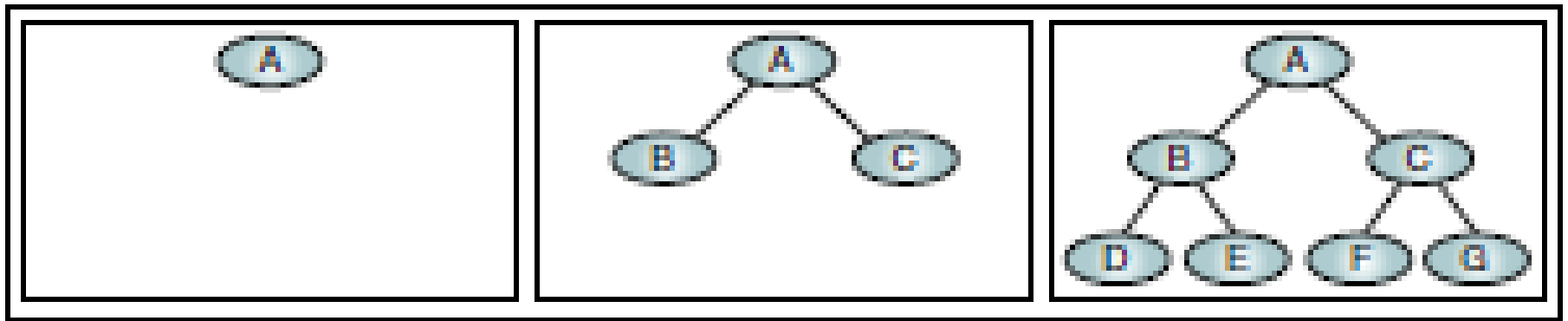
- Maximum number of nodes in a binary tree of height H
- If height of a binary tree is H , then
- Maximum number of nodes = $N_{\max} = 2^H - 1$
- Minimum number of nodes in a binary tree of height H
- If height of a binary tree is H , then
- Minimum number of nodes = $N_{\min} = H$

- Shorter the tree, it is easier to locate any desired node in the tree.
- Distance of a node from the root determines how efficiently it can be located.
- The children of any node in a tree can be accessed by following only one branch path, the one that leads to the desired node.
- The nodes at level 1 can be accessed by following only one branch.
- The nodes at level 2 can be accessed by following only two branches from the root.

- Balance is important characteristic of a binary tree.
- To determine whether a tree is balanced, it is needed to calculate balance factor.
- Balance Factor of a binary tree is the difference in height between its left and right subtrees.
- B is balance factor
- H_L is height of left sub-tree.
- H_R is height of right sub-tree.
- $B = H_L - H_R$

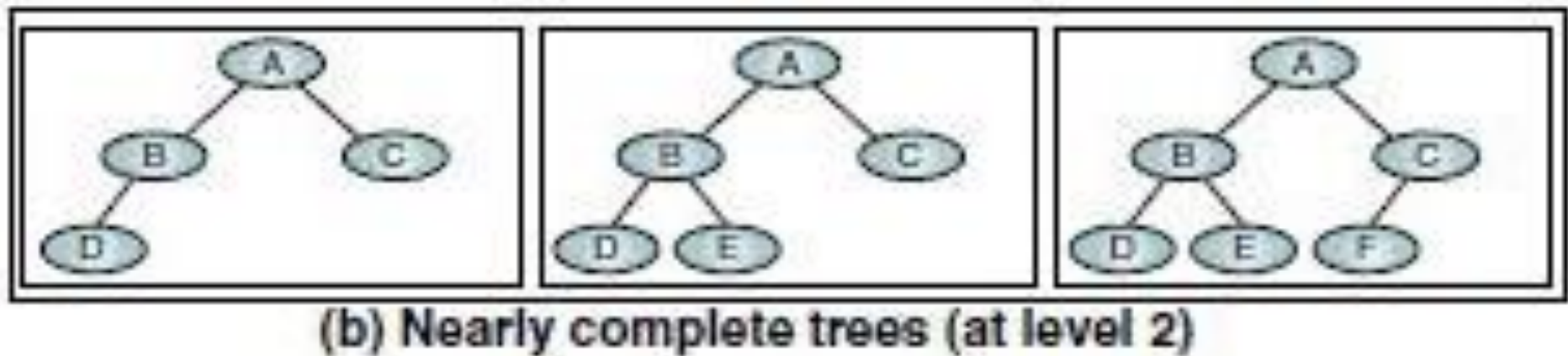
- **Balanced Binary Tree –**
- The height of its subtrees differ by no more than one.
- Its balance factor is -1 or 0 or $+1$ and its sub-trees are also balanced.

- Complete Binary Tree
- A complete tree has the maximum number of entries for its height.
- The maximum number is reached when the last level is full.



(a) Complete trees (at levels 0, 1, and 2)

- A nearly complete binary tree
- A tree is considered nearly complete, if it has the minimum height for its nodes and all nodes in the last level are found on the left.
- All the levels except last level are full of nodes and all nodes in the last level are found on the left.



Implementation of Binary Tree

- **Array (Sequential) Representation of Binary Trees**
- It uses a single linear array TREE
- The root (R) of the tree T is stored in TREE[1].
- If node N is present at TREE[K], then its left child is stored at TREE[2*K] and its right child is stored at TREE[2K+1].
- NULL is used to indicate an empty subtree.
- TREE[1]=NULL indicate empty tree.

- For a tree with depth d requires 2^{d+1} array size.
- E.g. $d=5$
- Number of nodes in complete binary tree = $2^d = 32$
- Array Size = $2^{d+1}=64$
- 32 locations needed extra to indicate empty subtrees.
- It is wastage of space.
- Sequential representation is inefficient unless the binary tree is complete or nearly complete.

Linked List Representation of Binary Tree

- Self referential structure can be used

Struct Node

```
{  
int data;  
struct node *left; struct node *right;  
}
```

Comparison

- Insertion and deletion is efficient in linked list
- Searching can be done efficiently in arrays.

Binary Tree Traversals

- **Traversal -**
- Each node of the tree is processed one and only once in a predetermined sequence.
- Two general approaches-
 - Depth-first Traversal.
 - Breadth-first Traversal

Depth-first Traversal

- It proceeds along a path from the root through one child to the most distant descendent of that first child before processing a second child.
- In depth-first traversal, we process all of the descendants of a child before going on to the next child.
- It proceeds vertically.
- Stack is used in Depth-first Traversal

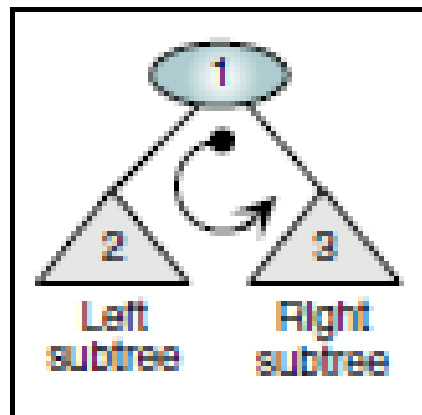
Breadth-first Traversal

- It proceeds horizontally from the root to all of its children, then to its children's children and so on till all the nodes have been processed.
- In breadth-first traversal, each level is completely processed before the next level is started.
- Queue is used in Breadth-first Traversal.

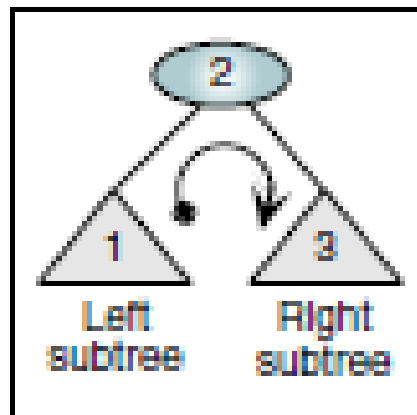
Depth-first Traversal

- A binary tree consists of a root, a left sub-tree and a right sub-tree
- Six different depth-first traversal sequences.
 - Root-Left-Right
 - Root-Right-Left
 - Left-Root-Right
 - Right-Root-Left
 - Left-Right-Root
 - Right-Left-Root

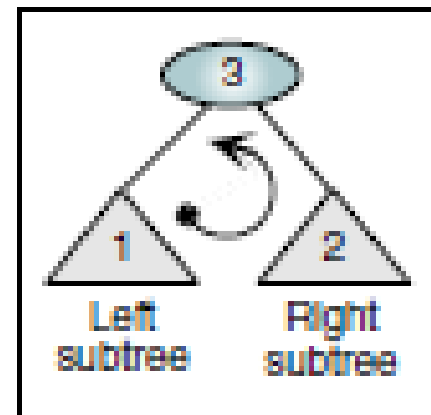
- In Computer Science three sequences are considered as standard.
- Process left node before right node
 - Root-Left-Right - Preorder Traversal (NLR)
 - Left-Root-Right - Inorder Traversal (LNR)
 - Left-Right-Root - Postorder Traversal



(a) Preorder traversal



(b) Inorder traversal



(c) Postorder traversal

Preorder Traversal

- Root-Left-Right - NLR
- Root is processed before its sub-trees

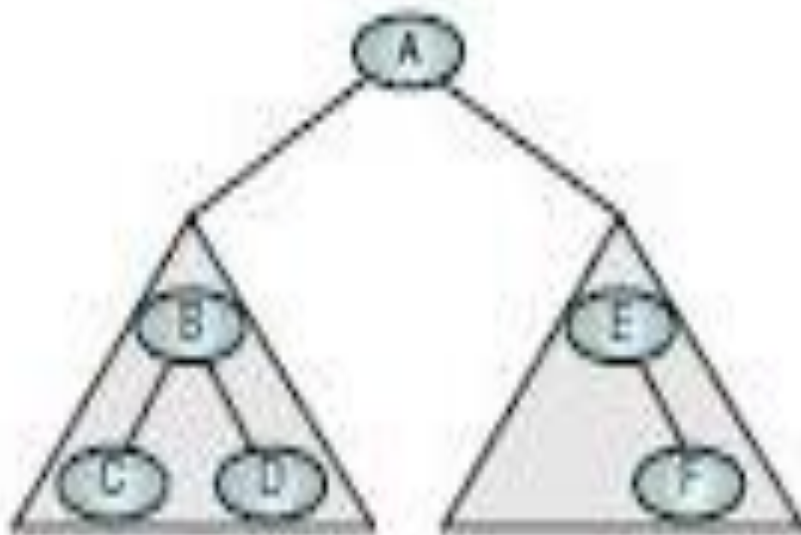
Algorithm preOrder (root)

Traverse a binary tree in node-left-right sequence.

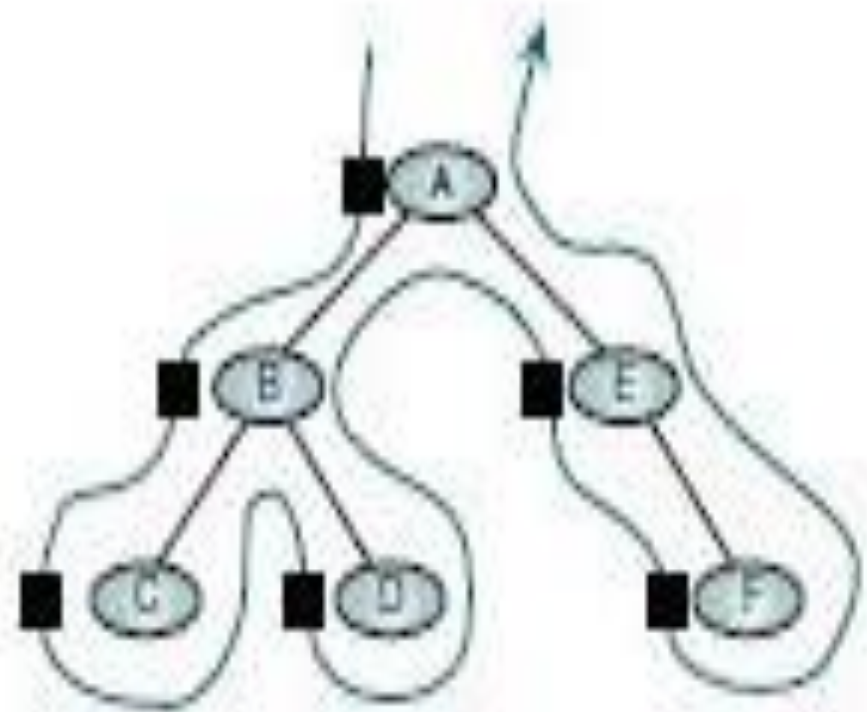
Pre root is the entry node of a tree or subtree

Post each node has been processed in order

```
1 if (root is not null)
  1 process (root)
  2 preOrder (leftSubtree)
  3 preOrder (rightSubtree)
2 end if
end preOrder
```



(a) Processing order



(b) "Walking" order

Inorder Traversal

- Left-Root-Right - LNR
- Root is processed between the sub-trees

Algorithm inOrder (root)

Traverse a binary tree in left-node-right sequence.

Pre root is the entry node of a tree or subtree

Post each node has been processed in order

1 if (root is not null)

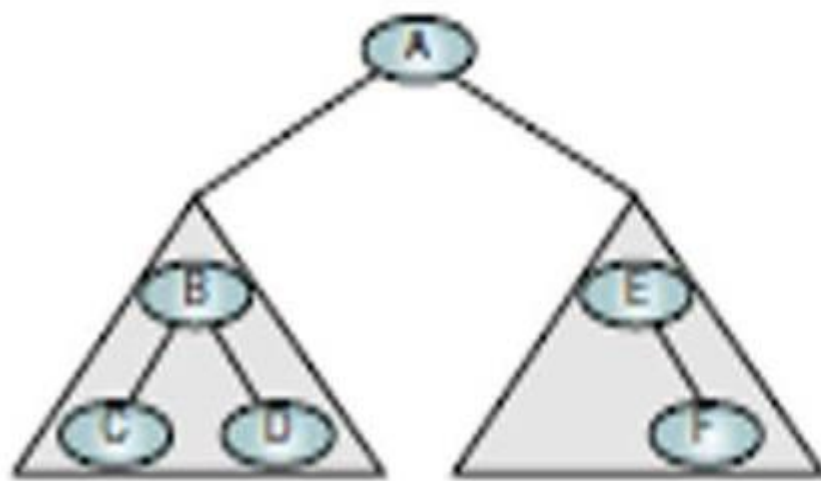
1 inOrder (leftSubTree)

2 process (root)

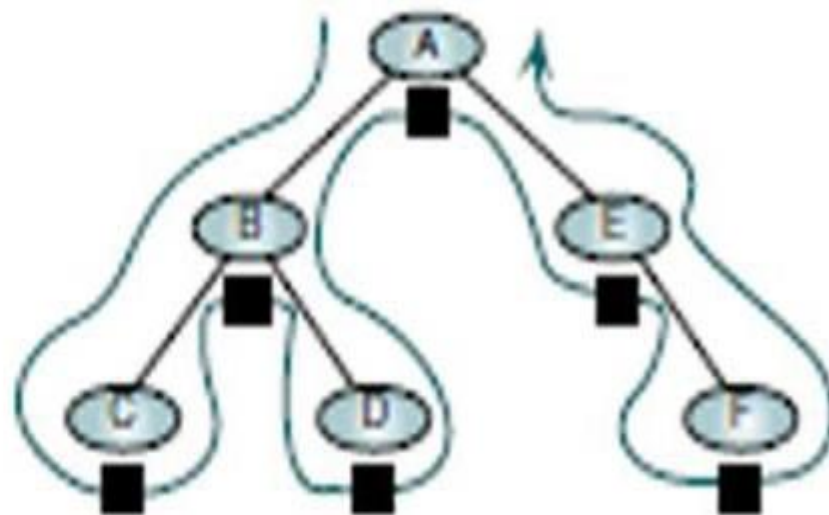
3 inOrder (rightSubTree)

2 end if

end inOrder



(a) Processing order



(b) "Walking" order

Postorder Traversal

- Left-Right-Root - LRN
- Root is processed after its sub-trees

```
Algorithm postOrder (root)
```

```
  Traverse a binary tree in left-right-node sequence.
```

```
    Pre  root is the entry node of a tree or subtree
```

```
    Post each node has been processed in order
```

```
1  if (root is not null)
```

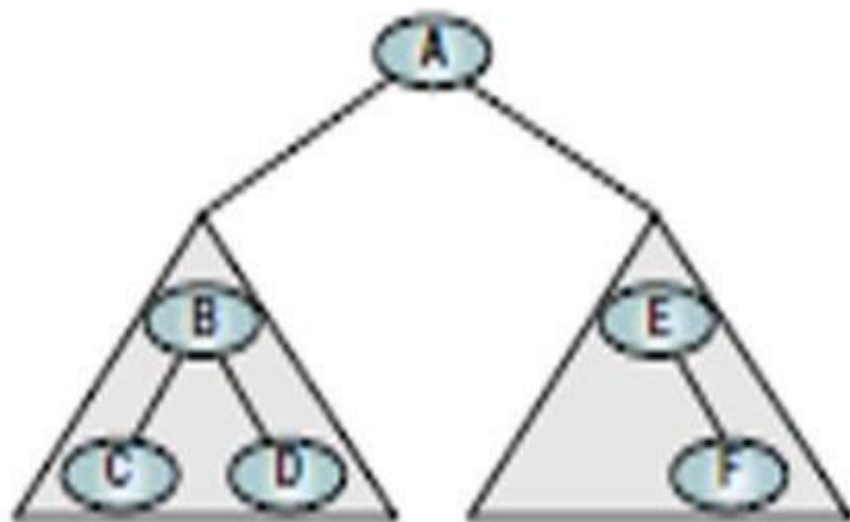
```
    1  postOrder (left subtree)
```

```
    2  postOrder (right subtree)
```

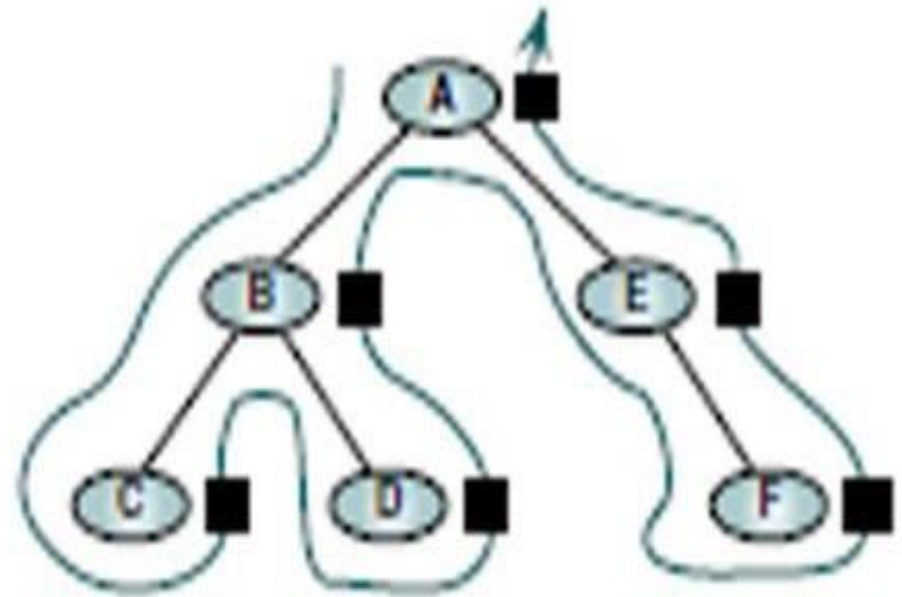
```
    3  process (root)
```

```
2  end if
```

```
end postOrder
```

(a) Processing order



(b) "Walking" order

Breadth-first Traversal

- It proceeds horizontally from the root to all of its children, then to its children's children and so on till all the nodes have been processed.
- In breadth-first traversal, each level is completely processed before the next level is started.
- Queue is used in Breadth-first Traversal.
- Given a node at level n , process all nodes at level n before proceeding with the nodes at level $n+1$.

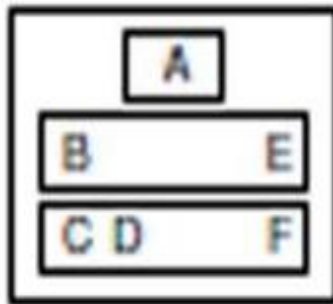
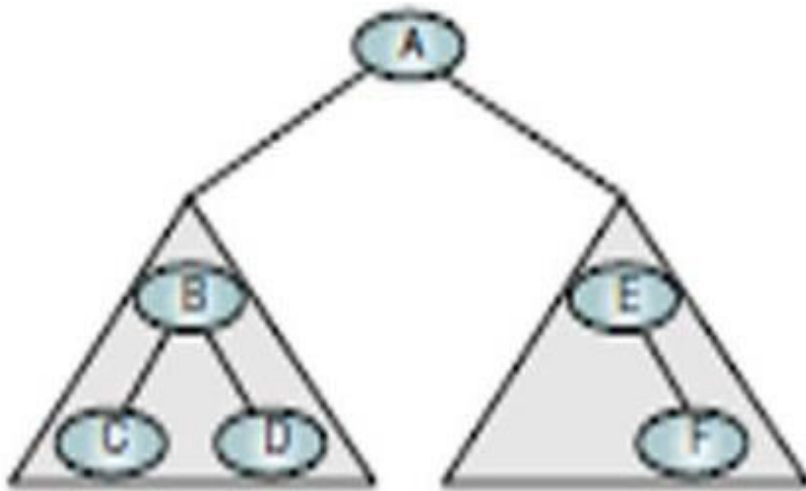
Algorithm breadthFirst (root)

Process tree using breadth-first traversal.

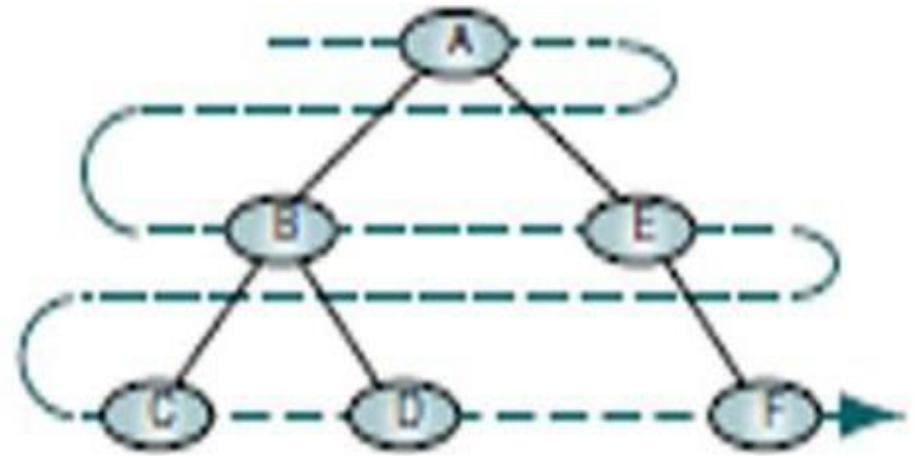
Pre root is node to be processed

Post tree has been processed

```
1 set currentNode to root
2 createQueue (bfQueue)
3 loop (currentNode not null)
  1 process (currentNode)
  2 if (left subtree not null)
    1 enqueue (bfQueue, left subtree)
  3 end if
  4 if (right subtree not null)
    1 enqueue (bfQueue, right subtree)
  5 end if
  6 if (not emptyQueue(bfQueue))
    1 set currentNode to dequeue (bfQueue)
  7 else
    1 set currentNode to null
  8 end if
4 end loop
5 destroyQueue (bfQueue)
end breadthFirst
```



(a) Processing order



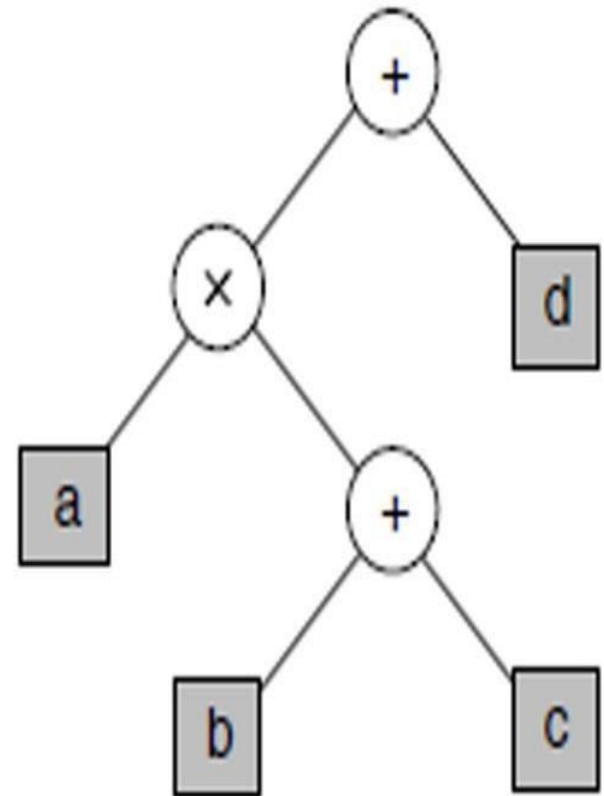
(b) "Walking" order

Expression Tree

- One of the application of binary tree.
- An expression is a sequence of tokens that follow prescribed rules.
- A token may be either an operand or an operator.
- Consider only binary arithmetic operators in the form operand–operator–operand.
- The standard arithmetic operators are +, -, *, and /

- An expression tree is a binary tree with the following properties:
- Each leaf is an operand.
- The root and internal nodes are operators.
- Sub-trees are sub-expressions, with the root being an operator.

$a \times (b + c) + d$



Infix Expression and Its Expression Tree

- For an expression tree, the three standard depth-first traversals represent the three different expression formats: infix, postfix, and prefix.
- The inorder traversal produces the infix expression.
- The preorder traversal produces the prefix expression.
- The postorder traversal produces the postfix expression.

Infix Traversal of an Expression Tree

- While traversing the infix expression tree, add an opening parenthesis at the beginning of each expression and a closing parenthesis at the end of each expression.
- The root of the tree and each of its subtrees represents a subexpression,
- Add the opening parenthesis when start a tree or a subtree and
- The closing parenthesis when processed all of its children.

Infix Expression Tree Traversal

Algorithm infix (tree)

Print the infix expression for an expression tree.

Pre tree is a pointer to an expression tree

Post the infix expression has been printed

1 if (tree not empty)

1 if (tree token is an operand)

1 print (tree-token)

2 else

1 print (open parenthesis)

2 infix (tree left subtree)

3 print (tree token)

4 infix (tree right subtree)

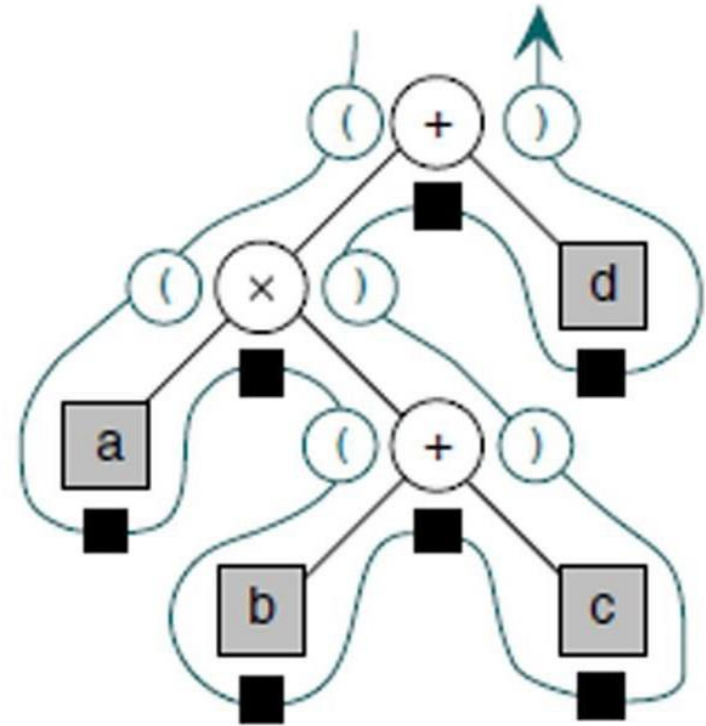
5 print (close parenthesis)

3 end if

2 end if

end infix

$((a \times (b + c)) + d)$



Infix Traversal of an Expression Tree

Prefix Traversal of an Expression Tree

- It uses the standard preorder tree traversal.
- No parentheses are necessary to add

Algorithm prefix (tree)

Print the prefix expression for an expression tree.

Pre tree is a pointer to an expression tree

Post the prefix expression has been printed

```
1 if (tree not empty)
  1 print (tree token)
  2 prefix (tree left subtree)
  3 prefix (tree right subtree)
2 end if
end prefix
```

Postfix Traversal of an Expression Tree

- The postfix traversal of an expression uses the basic postorder traversal of binary tree.
- It does not require parentheses to add.

Algorithm postfix (tree)

Print the postfix expression for an expression tree.

Pre tree is a pointer to an expression tree

Post the postfix expression has been printed

1 if (tree not empty)

1 postfix (tree left subtree)

2 postfix (tree right subtree)

3 print (tree token)

2 end if

end postfix

Huffman Code

- The American Standard Code for Information Interchange (ASCII) uses 7 bits to represent all characters.
- It do not consider frequency of occurrence.
- If frequency is more, it should be encoded in short.

- Steps

1] First organize the entire character set into a row, ordered according to frequency from highest to lowest (or vice versa).

Each character is now a node at the leaf level of a tree.

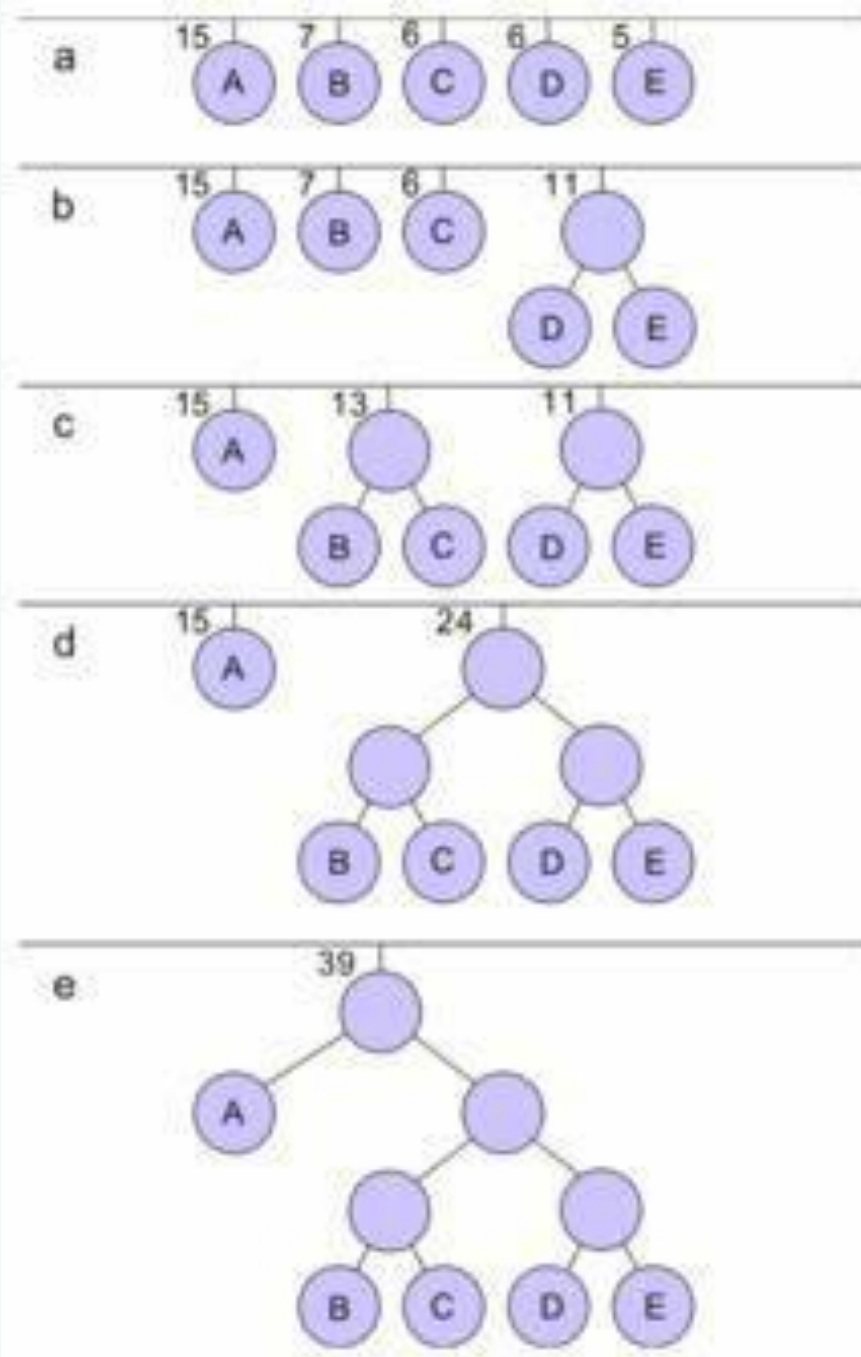
2] Find the two nodes with the smallest combined frequency weights and join them to form a third node, resulting in a simple two-level tree.

The weight of the new node is the combined weights of the original two nodes.

This node, one level up from the leaves, is eligible to be combined with other nodes.

The sum of the weights of the two nodes chosen must be smaller than the combination of any other possible choices.

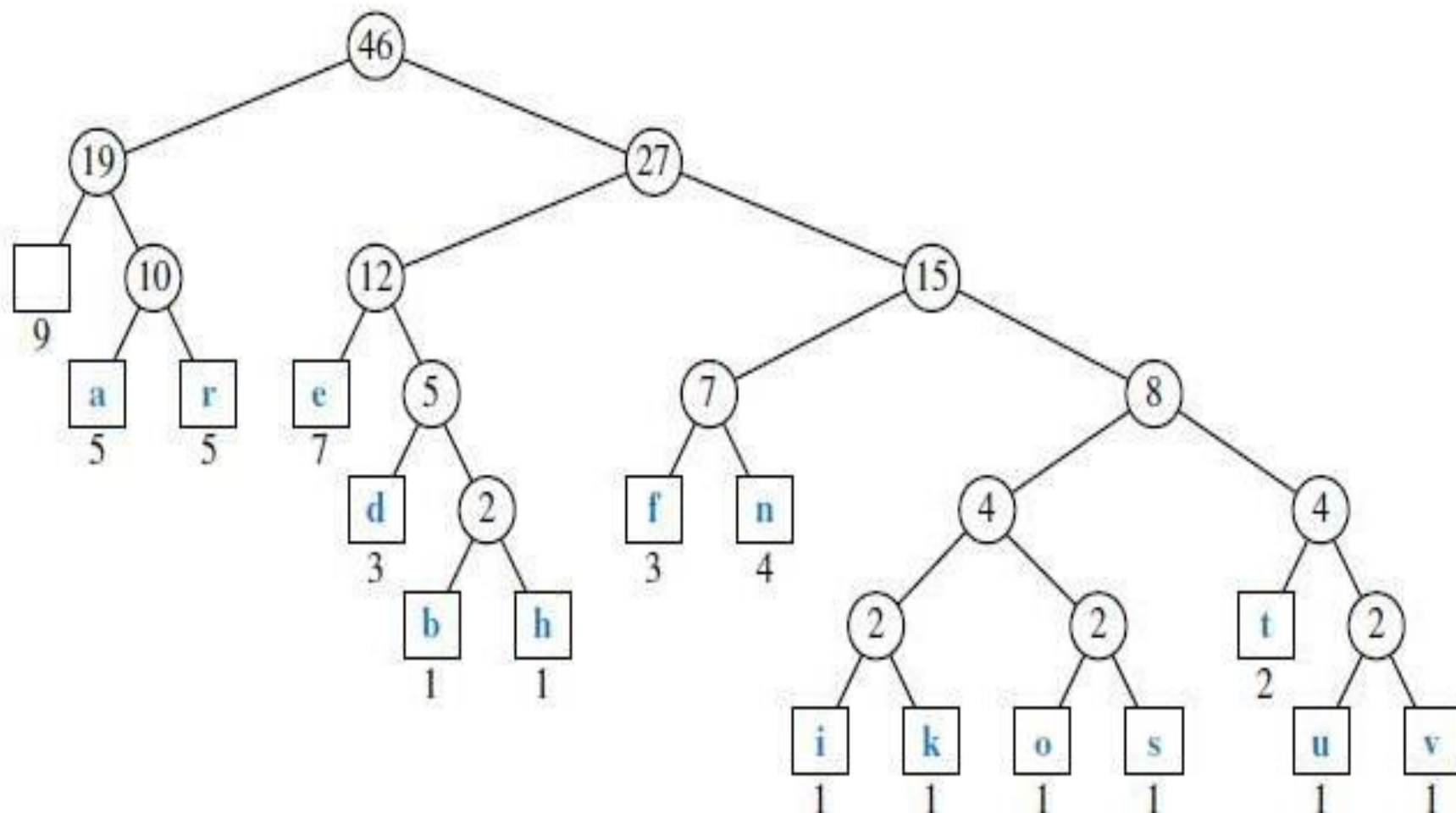
3] Repeat step 2 until all of the nodes, on every level, are combined into a single tree.



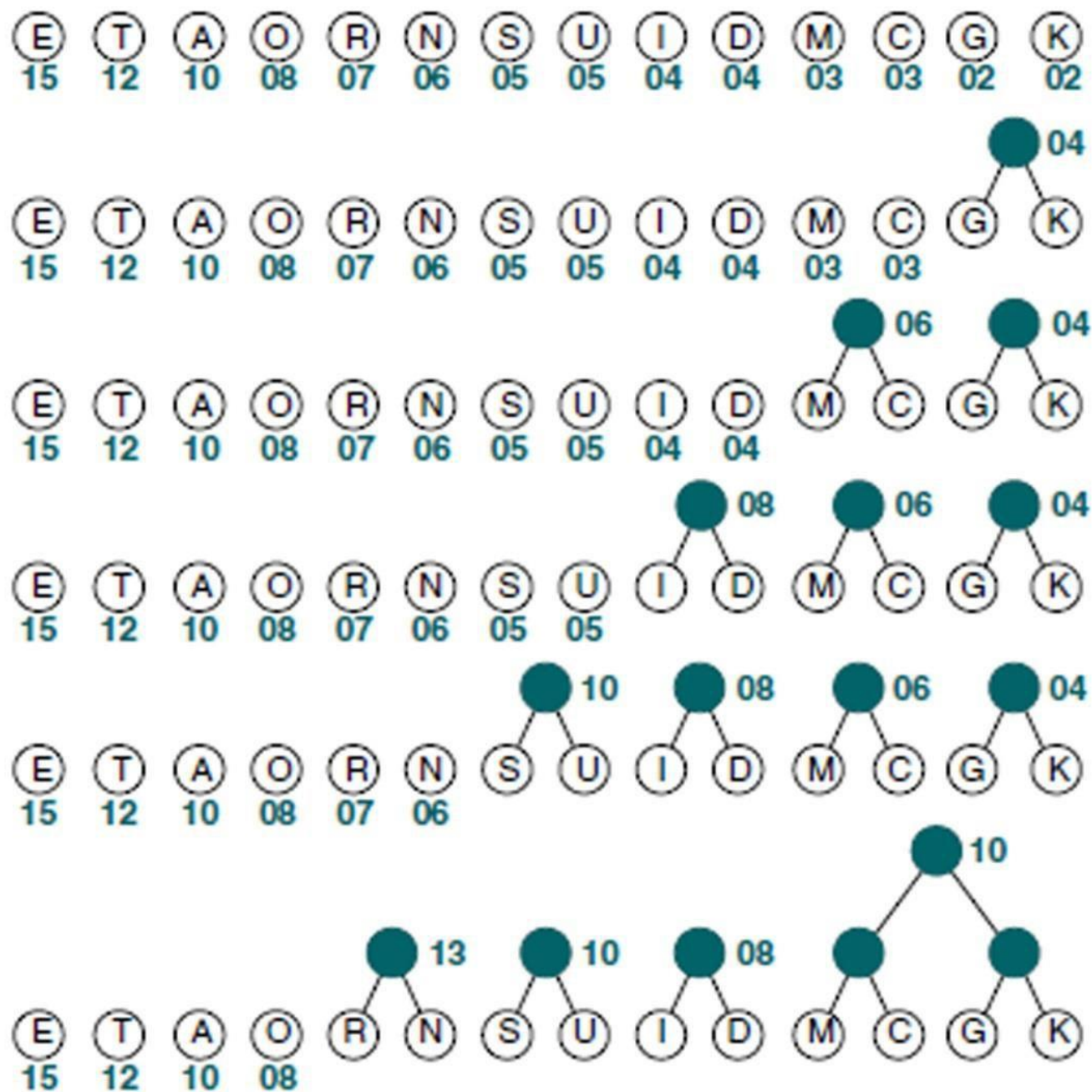
(a)

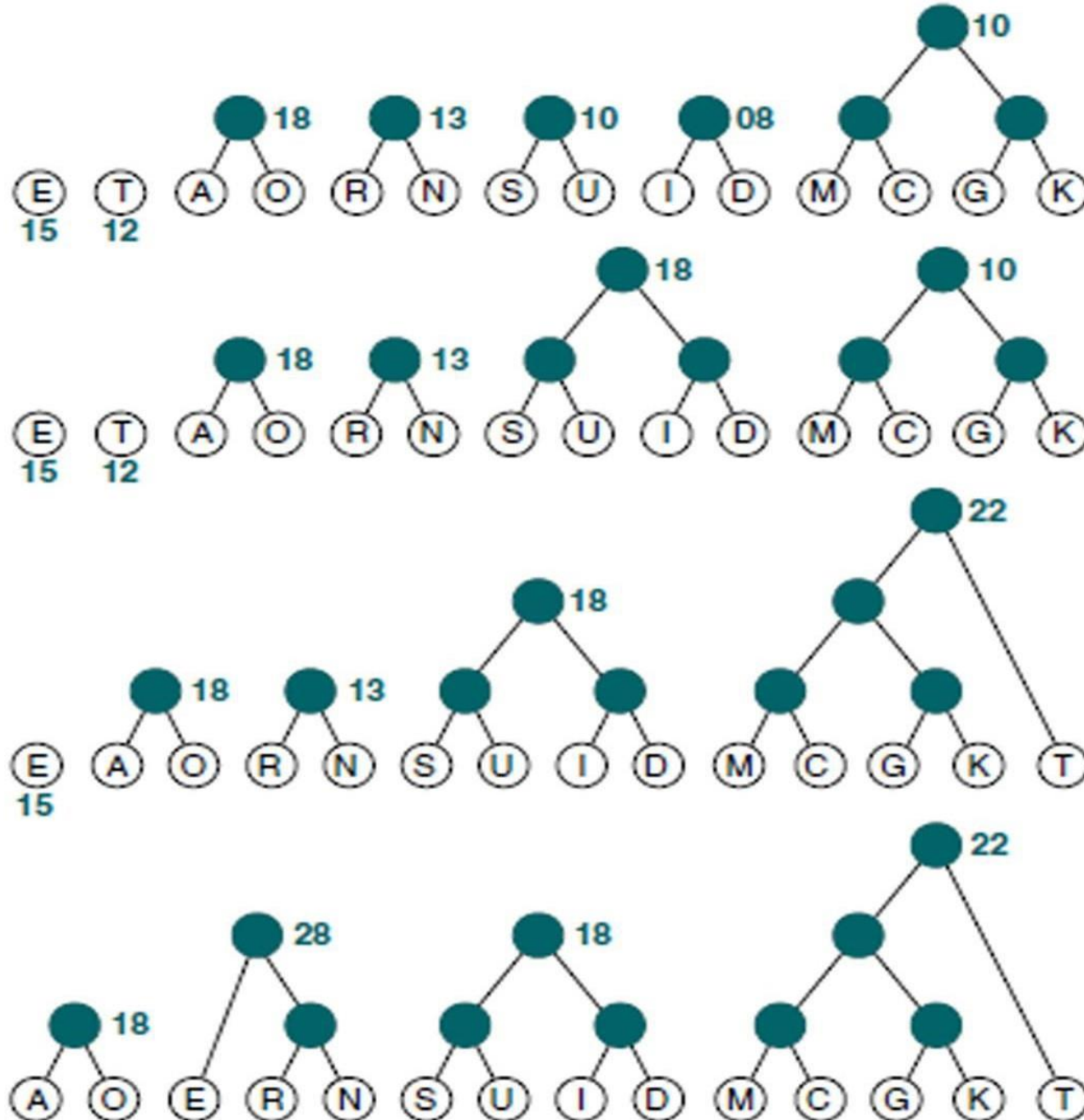
Character		a	b	d	e	f	h	i	k	n	o	r	s	t	u	v
Frequency	9	5	1	3	7	3	1	1	1	4	1	5	1	2	1	1

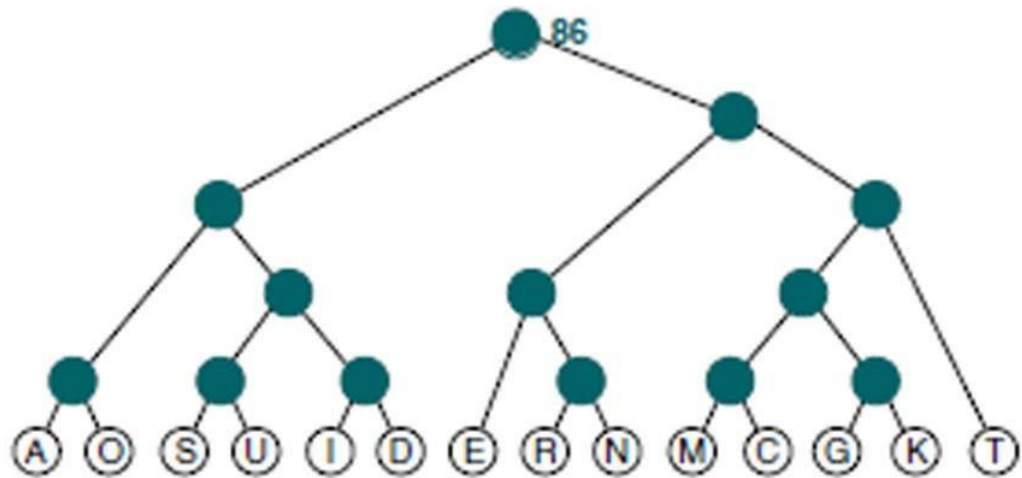
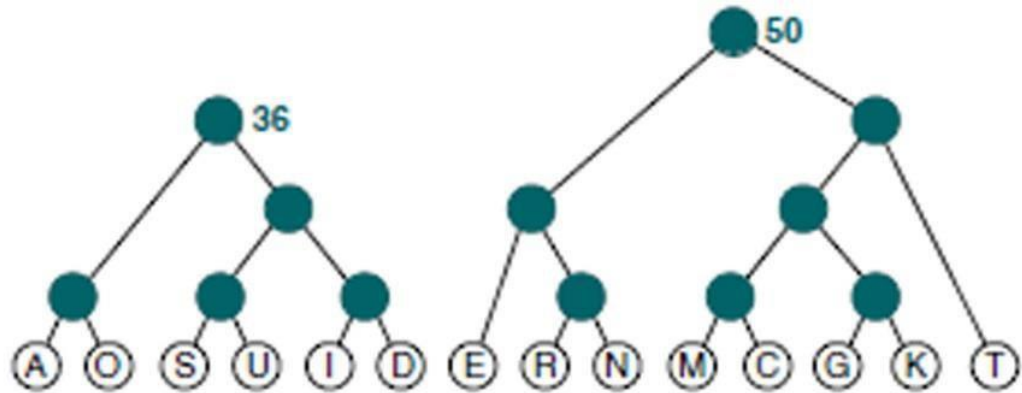
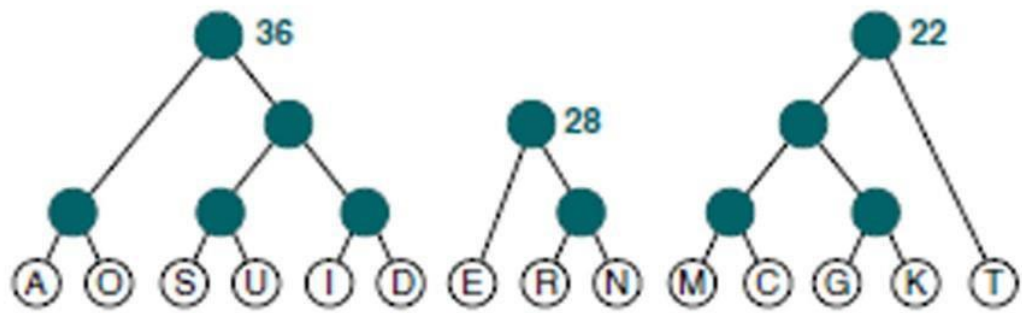
(b)

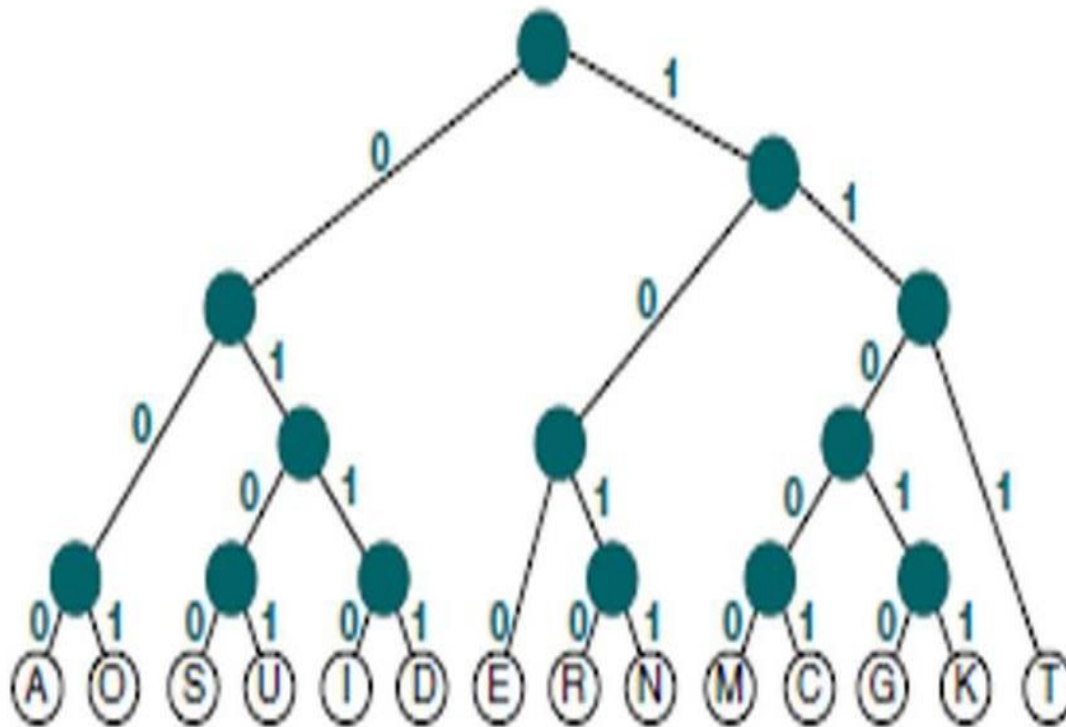


Character	Weight	Character	Weight	Character	Weight
A	10	I	4	R	7
C	3	K	2	S	5
D	4	M	3	T	12
E	15	N	6	U	5
G	2	O	8		









A = 000	U = 0101	E = 100	M = 11000	K = 11011
O = 001	I = 0110	R = 1010	C = 11001	T = 111
S = 0100	D = 0111	N = 1011	G = 11010	

- Huffman code is widely used for data compression; it reduces the number of bits sent or stored.

-

000110100010010111100000010101101110011

1

- After receiving the first bit, start from the root and follow the 000 path as we read the next two bits, arriving at the leaf A.
- Because a leaf is reached, we have decoded the first character.

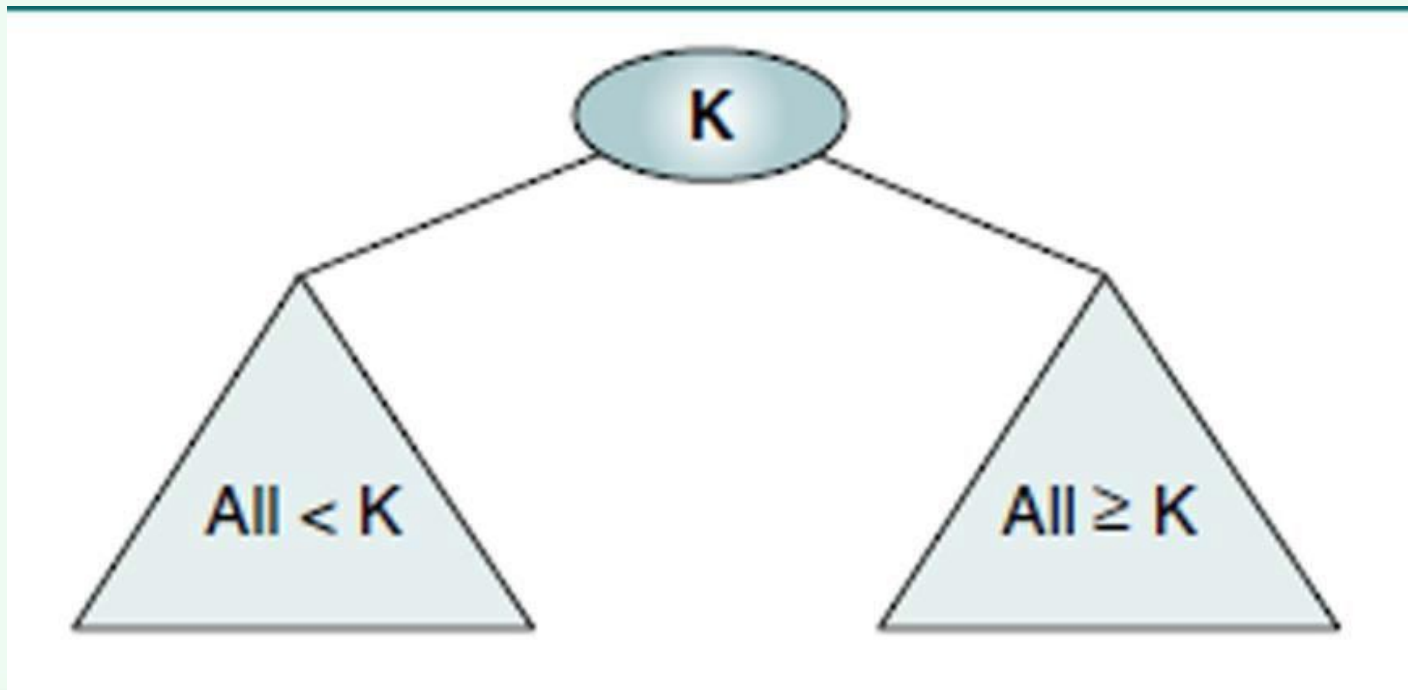
• Then start the next character with 1 and, starting with

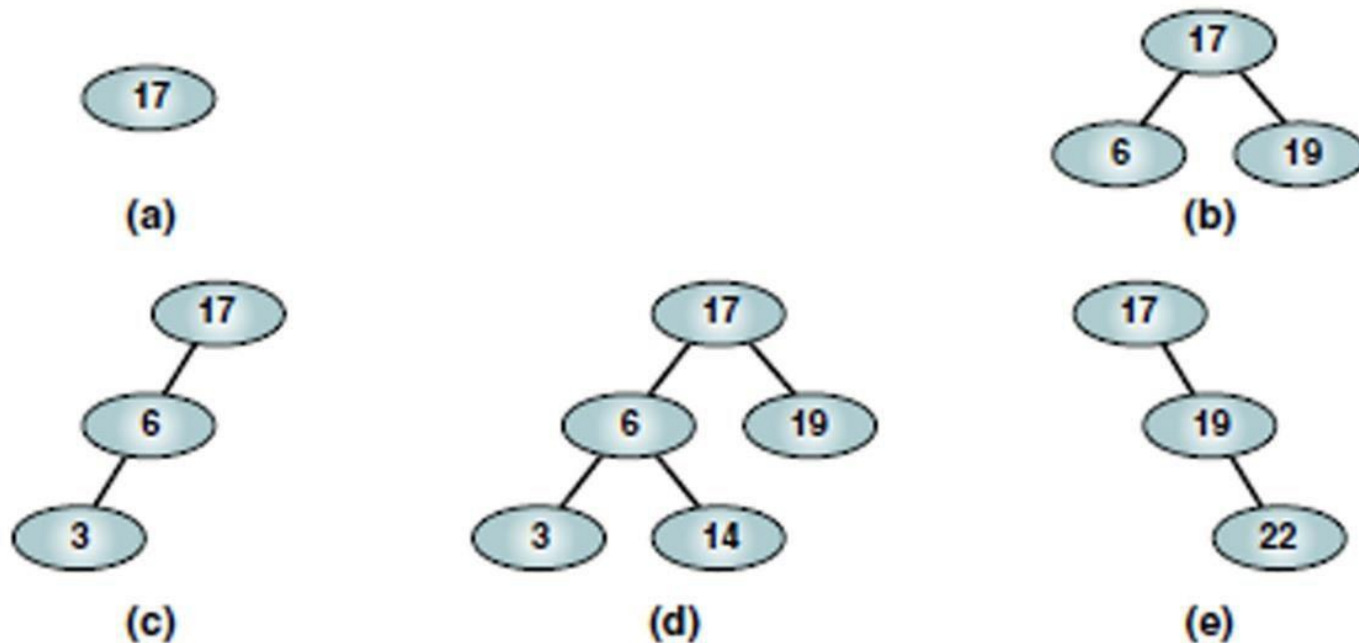
- we start at the root again until all of the message is received.
- Because no spaces in our code, the words are run together.
- AGOODMARKET

Binary Search Tree

- A binary search tree (BST) is a binary tree with the following properties:
- All items in the left subtree are less than the root.
- All items in the right subtree are greater than or equal to the root.
- Each subtree is itself a binary search tree.

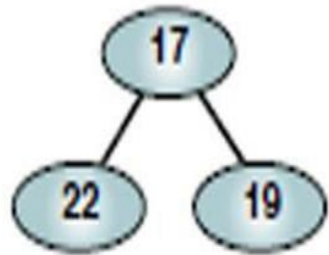
- The information represented by each node is a record rather than a single data element.
- In a binary search tree, the left subtree contains key values less than the root and the right subtree contains key values greater than or equal to the root.



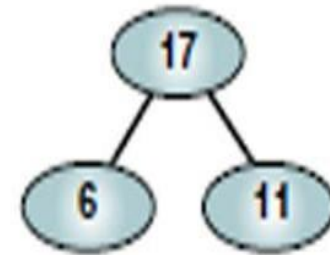


Valid Binary Search Trees

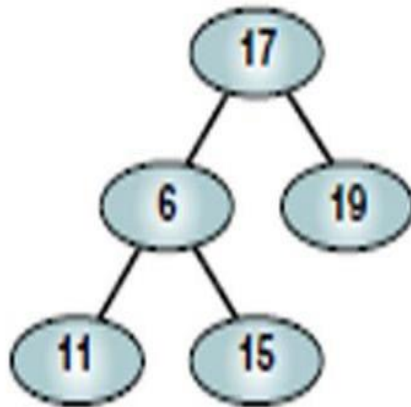
- Trees (a) and (b) are complete and balanced
- Tree (d) is nearly complete and balanced
- Trees (c) and (e) are neither complete nor balanced.



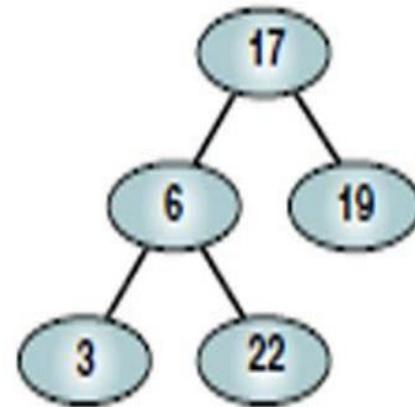
(a)



(b)



(c)



(d)

Invalid Binary Search Trees

Binary Search Tree ADT

- Data

Key/Data/Information of a Node

Address of Left Child (sub-tree)

Address of Right Child (sub-tree)

- **Operations**
- **Traverse(root)**
Visit all the nodes of tree in particular order
- **Search(root, skey)**
Search for a node having data skey
- **Insert(root, newnode)**
Insert new node at its proper position in binary search tree
- **Delete(root, dkey)**
Remove node having data dkey

Binary Search Tree Operations

- Traversals
- Search
- Insertion
- Deletion

Preorder Traversal

- Root-Left-Right - NLR
- Root is processed before its sub-trees

Algorithm preOrder (root)

Traverse a binary tree in node-left-right sequence.

Pre root is the entry node of a tree or subtree

Post each node has been processed in order

```
1 if (root is not null)
  1 process (root)
  2 preOrder (leftSubtree)
  3 preOrder (rightSubtree)
2 end if
end preOrder
```

Inorder Traversal

- Left-Root-Right - LNR
- Root is processed between the sub-trees

Algorithm inOrder (root)

Traverse a binary tree in left-node-right sequence.

Pre root is the entry node of a tree or subtree

Post each node has been processed in order

1 if (root is not null)

1 inOrder (leftSubTree)

2 process (root)

3 inOrder (rightSubTree)

2 end if

end inOrder

Postorder Traversal

- Left-Right-Root - LRN
- Root is processed after its sub-trees

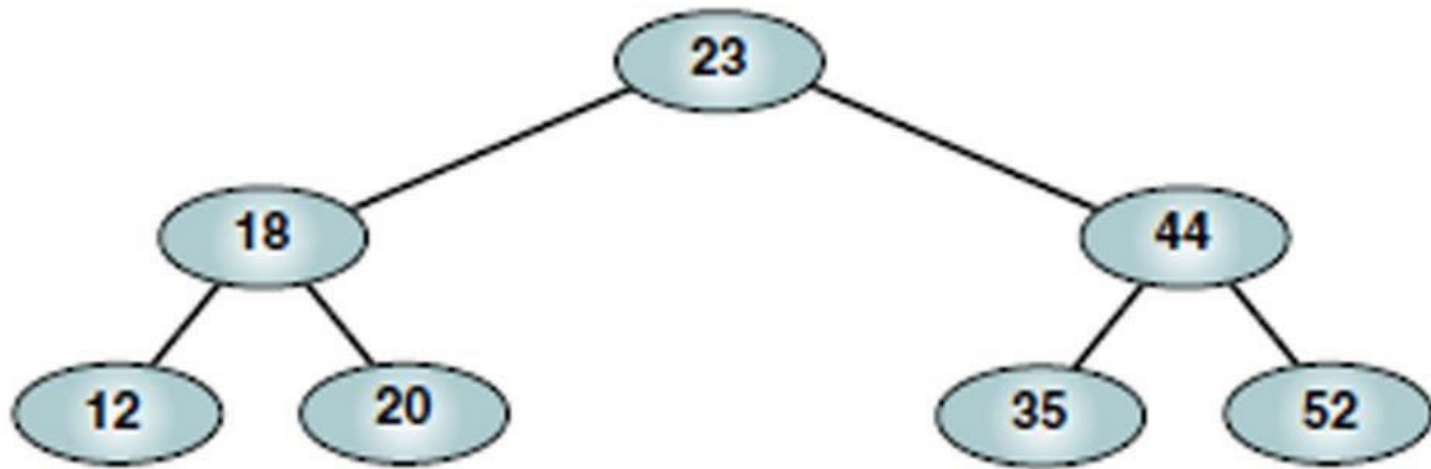
```
Algorithm postOrder (root)
```

```
Traverse a binary tree in left-right-node sequence.
```

```
Pre root is the entry node of a tree or subtree
```

```
Post each node has been processed in order
```

```
1 if (root is not null)
  1 postOrder (left subtree)
  2 postOrder (right subtree)
  3 process (root)
2 end if
end postOrder
```



- Preorder – 23 18 12 20 44 35 52
- Inorder – 12 18 20 23 35 44 52
- Postorder – 12 20 18 35 52 44 23

Find Smallest Node in BST

Algorithm findSmallestBST (root)

This algorithm finds the smallest node in a BST.

Pre root is a pointer to a nonempty BST or subtree

Return address of smallest node

1 if (left subtree empty)

 1 return (root)

2 end if

3 return findSmallestBST (left subtree)

end findSmallestBST

Find Largest Node in BST

Algorithm findLargestBST (root)

This algorithm finds the largest node in a BST.

Pre root is a pointer to a nonempty BST or subtree

Return address of largest node returned

1 if (right subtree empty)

1 return (root)

2 end if

3 return findLargestBST (right subtree)

end findLargestBST

Search

Algorithm searchBST (root, targetKey)

Search a binary search tree for a given value.

Pre root is the root to a binary tree or subtree

 targetKey is the key value requested

Return the node address if the value is found

 null if the node is not in the tree

1 if (empty tree)

 Not found

 1 return null

2 end if

3 if (targetKey < root)

 1 return searchBST (left subtree, targetKey)

4 else if (targetKey > root)

 1 return searchBST (right subtree, targetKey)

5 else

 Found target key

 1 return root

6 end if

end searchBST

Insert Node - Add Node

- Insertion of new node at leaf only.

Algorithm addBST (root, newNode)

Insert node containing new data into BST using recursion.

Pre root is address of current node in a BST

 newNode is address of node containing data

Post newNode inserted into the tree

Return address of potential new tree root

1 if (empty tree)

1 set root to newNode

2 return newNode

2 end if

Locate null subtree for insertion

3 if (newNode < root)

1 return addBST (left subtree, newNode)

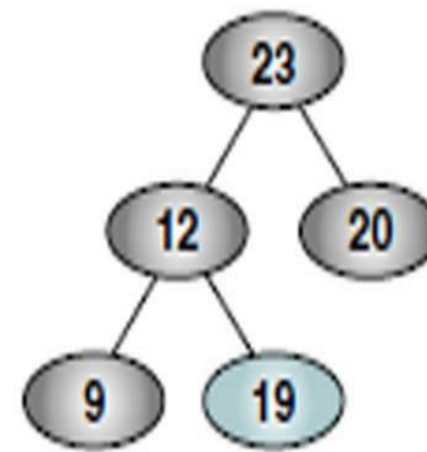
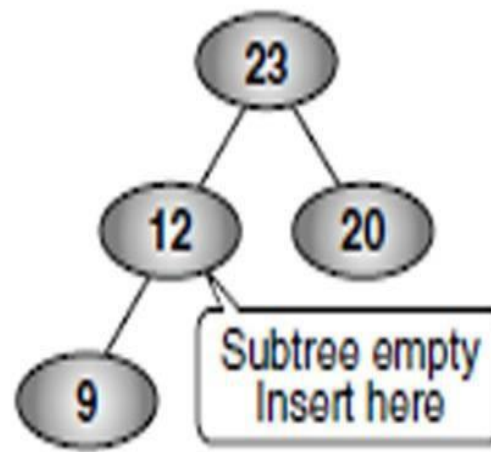
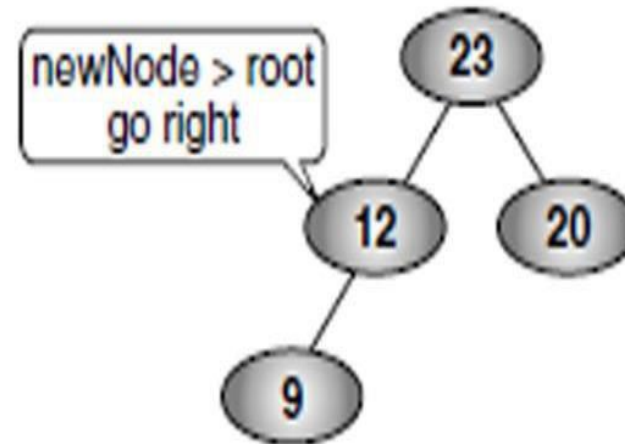
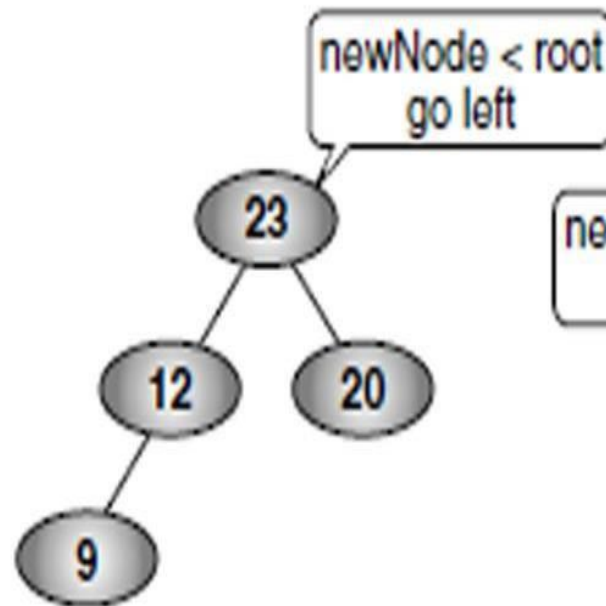
4 else

1 return addBST (right subtree, newNode)

5 end if

end addBST

19
newNode



Delete Node

- To delete a node from a binary search tree, we must first locate it.
- There are four possible cases when we delete a node:
 1. The node to be deleted has no children. In this case,
all we need to do is delete the node.
 2. The node to be deleted has only a right subtree.
We delete the node and attach the right subtree to the deleted node's parent.

3.The node to be deleted has only a left subtree. We delete the node and attach the left subtree to the deleted node's parent.

4.The node to be deleted has two subtrees.

It is possible to delete a node from the middle of a tree, but the result tends to create very unbalanced trees.

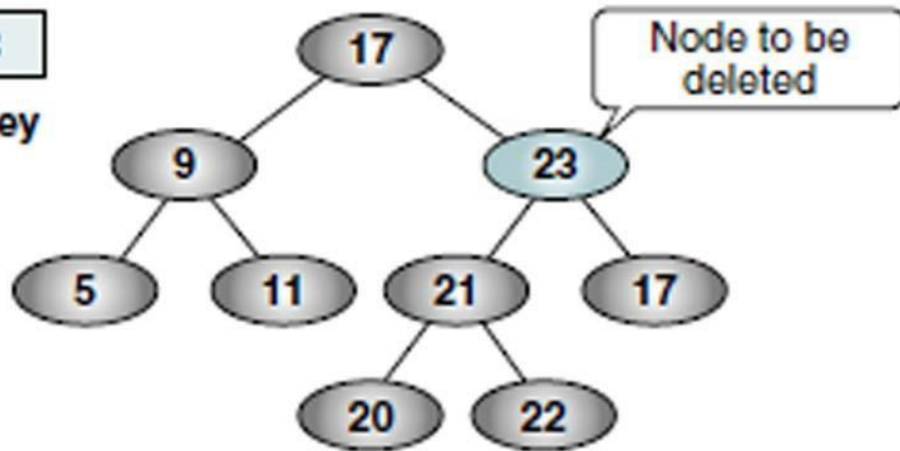
Rather than simply delete the node, therefore, we try to maintain the existing structure as much as possible by finding data to take the place of the deleted data.

- This can be done in one of two ways:
 - (1) we can find the largest node in the deleted node's left subtree and move its data to replace the deleted node's data or
 - (2) we can find the smallest node on the deleted node's right subtree and move its data to replace the deleted node's data.
- Regardless of which logic we use, we will be moving data from a leaf or a leaflike node that can then be deleted.

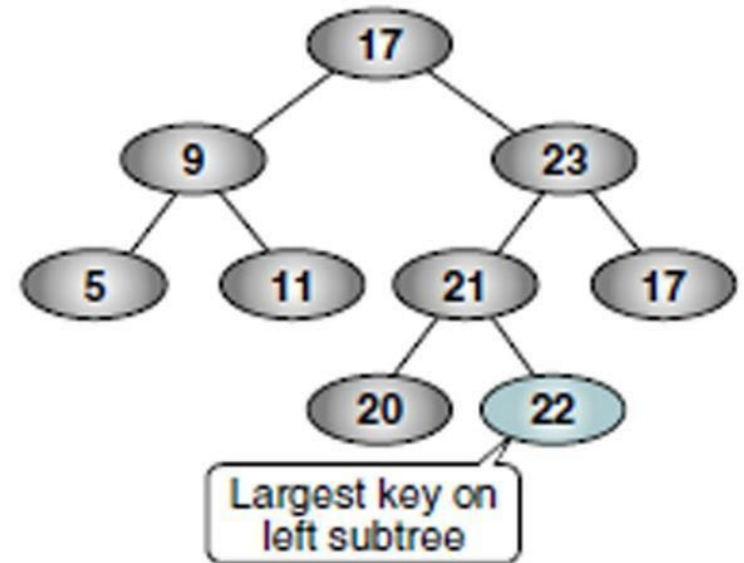
```
end deleteBST
```


23

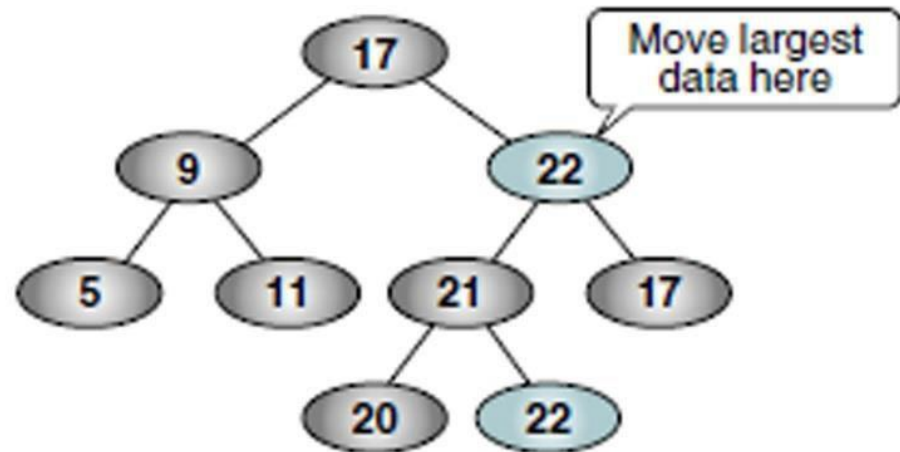
dltKey



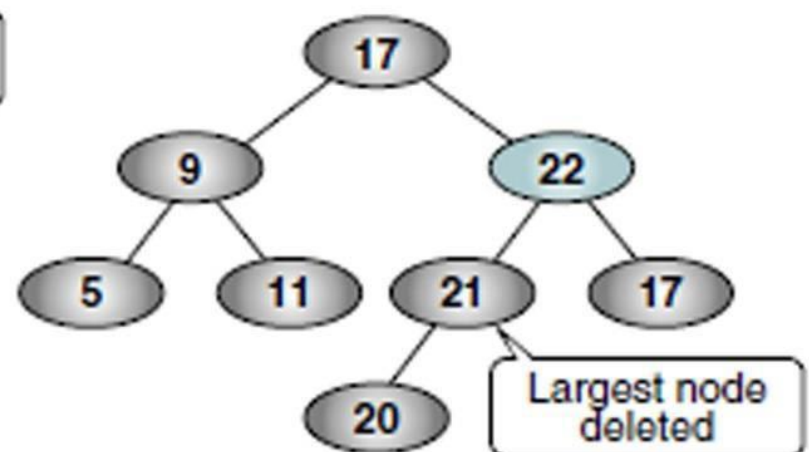
(a) Find dltKey



(b) Find largest



(c) Move largest data



(d) Delete largest node

Thank You